

黑鸚鵡:一个敏捷的 开源RISC-V多核加 速器soc

Daniel Petrisko, Farzam Gilani, Mark Wyse, Dai Cheol
Jung, Scott Davidson, Paul Gao, 和Chun Zhao

华盛顿大学

Zahra Azad和Sadullah Canakci

波士顿大学的bandh

av Veluri和Tavio Guarino

华盛顿大学

Ajay Joshi

波士顿大学

马克·奥斯卡和迈克尔·贝德福德·泰勒

华盛顿大学

摘要:本文介绍了黑鸚鵡, 它的目标是成为世界上默认的开源, 支持linux, 缓存一致, 64位RISC-V多核。在实现这一目标的过程中, 我们的研究旨在推进世界对“硬件的软件工程”的认识。虽然最初是由华盛顿大学和波士顿大学在DARPA的资助下启动的, 但黑鸚鵡致力于社区驱动和基础设施不可知;一个在功率、性能、面积和复杂性方面都是帕累托最优的多核系统。为了确保黑鸚鵡易于使用, 扩展, 最重要的是, 信任, 开发遵循三个核心原则:小型化, 模块化和友好化。开发工作优先考虑使用意向接口和模块化以及硅验证作为一阶设计指标, 以便用户可以快速入门并相信他们的设计将执行为

数字对象标识符10.1109/MM.2020.2996145

出版日期2020年5月20日;当前版本日期2020年6月30日。

预计部署时。黑鸚鵡已在GlobalFoundries的12纳米FinFET纤带中得到验证。黑鸚鵡是理想的独立Linux处理器，或作为灵活加速器SoC设计流程的可伸缩结构。

& RISC-V9是一项颠覆性技术。从未有过如此庞大的全球社区共同努力，提出完整的开源指令集、机器模型和软件堆栈。社区中有一种强烈的信念，即RISC-V将作为低功耗、高性能的主机核心，在敏捷开发的专用加速器中找到自己的立足点，这些加速器是为了响应Dennard Scaling的终结而创建的。

然而，奇怪的是，缺乏类似的全球维护的RISC-V SoC的开源实现。本文中描述的黑鸚鵡旨在填补这一空白。黑鸚鵡是开源的，现在在BSD-3许可下可用。黑鸚鵡是用标准的SystemVerilog编写的，因此可以毫不费力地集成到现有的设计方法中，并且很容易被工业设计师理解和修改。黑鸚鵡是在GlobalFoundries的12纳米工艺中制造的，经过几次迭代重构，以获得高面积、延迟和功率效率。

从软件工程和Linux开发的可扩展性中吸取教训，黑鸚鵡采用模块化设计，将接口放在首位。我们相信，这将允许贡献者独立工作，而不必完全了解黑鸚鵡的所有组件，或者它们是如何发展的，从而实现可扩展的协作。我们不仅寻求在开源处理器架构中推进最先进的技术，还寻求开发改变世界设计硬件方式的方法。

成功度量标准

我们相信黑鸚鵡的采用将由四个方面的优化驱动，如图1所示：质量、病毒式传播、功能和效率。

质量：从一开始，黑鸚鵡的架构就不仅仅是为了实现RISC-V的遵从性，而是为了产生一个高质量的代码库，工程师们可以从本质上信任它作为一个安全和高质量的设计。这种方法（我们非正式地称之为硬件的软件工程）的核心是普遍使用有意设计的、狭窄的、模块化的接口，这些接口使设计易于推理，而不会牺牲性能、功率或面积。利用多年的处理器设计经验，我们开发了一个高级设计文档，将多核划分为三个主要模块，具有易于理解的轻量级事务性接口。然后每个模块都有自己的设计文档，其中指定了自己的内部模块接口，并处理了重要的细微差别和特殊情况。从那里，我们产生了原理图，然后是利用BaseJump标准模板库(STL)的SystemVerilog RTL，⁸是一组为通用计算机体系结构和硬件原子精心设计的扩展接口，附带相应的硅验证的SystemVerilog实现。然后，RTL进行了广泛的代码审查，以及单元和随机测试。我们正在系统地测量切换、行和功能覆盖，并在每天的基础上推动我们的验证方法的覆盖。我们的目标是让有经验的工程师能够评估文档、代码和测试；欣赏黑鸚鵡的品质；并自信地使用它。

病毒式传播：虽然黑鸚鵡是一个高质量的设计，但我们意识到，如果它不被广泛采用，如果社区不集体管理它，它最终将是不成功的。出于这个原因，我们一直专注于开箱即用的体验，让它简单地起床和

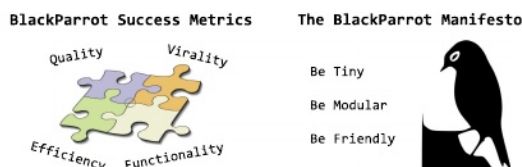


图1. 黑鸚鵡成功指标和宣言。成功指标在战略上使项目得到广泛采用，黑鸚鵡宣言为技术决策提供了战术指导。

	CoreMark/MHz (Higher better)	Language	Design Schema	Natively Multicore	License
BlackParrot	3.04	SystemVerilog	Pervasive Modular Interfaces; Standard Template Library	✓	BSD-3
Rocket	2.62	Chisel	Generator	✓	BSD-3
Ariane	2.45	SystemVerilog	Monolithic		SHL-2
Shakti	2.20	BlueSpec	Generator		BSD-3
Flute	1.00	BlueSpec	Generator		Apache-2
SiFive U54MC	3.01	Chisel	Generator	✓	Closed Source

图2。最新的能源/性能优化64位RISC-V开源linux支持ASIC应用类处理器内核。每核性能以CoreMark/MHz作为行业标准给出。作为参考，我们给出了等效的闭源，付费，linux支持的SiFive U54内核。(注意，由于这些开源项目都是活的、会呼吸的项目，所以这只是一个及时的快照!)

通过github checkout运行，并尽可能少地拉入外部组件。我们使用一种广为人知的语言，SystemVerilog，而不是Chisel或BlueSpec。在我们的社交互动中，我们注重友好和包容。太多的在线协作论坛因容忍辱骂行为而受到损害，尤其是社区中受人尊敬的成员，最近的一个案例突出了这一点:Linus Torvalds自己离开Linux几个月，试图思考他的坏脾气态度所带来的不良影响。我们积极尝试防止“非我发明”综合症在黑鸚鵡的努力中占据主导地位。我们的努力欢迎您的贡献，设计的模块化特性使个人贡献者很容易跟上速度。

功能:黑鸚鵡启动Linux。它实现了当前软件栈使用的RISC-V指令集的所有核心功能。它还包含了现代soc所需的所有有用功能，如中断控制器、相干缓存层次结构和易于集成的加速器接口。

效率:BlackParrot必须在其目标领域拥有一流的PPA;用于加速器芯片的Linux主机多核。早期的Coremark分数显示，BlackParrot在其班级的学术和商业核心方面都取得了具有竞争力的表现，如图2所示。广泛的设计和RTL代码审查被用来确保BlackParrot包含一个linux类微处理器所期望的现代微架构功能的有效实现。SRAM和逻辑结构的大小符合

性能、功率和面积(PPA)效率。为了验证该设计，我们在GlobalFoundries的12纳米工艺节点上制造了BlackPar-rot(见图4)，并在频繁的12纳米和40纳米tapeouts上部署了新功能。

设计宣言

在项目的过程中，有很多情况下，有两个合理的技术方向来采取项目。为了指导我们的工作，我们制定了一个非正式的宣言来帮助决定这些困难的追逐。宣言有以下三条关键规则。

- 1)小一点。在选择备选方案时，我们会选择能够产生更小、更容易理解的代码库、更少死亡区域、更简单关键路径和更少bug的选项。其结果是代码库尽可能小且易于理解，硬件是PPA高效的。我们注意不要在RTL中实现深奥和性能非关键的组件，并避免最近基于生成器的RTL方法中的一个常见问题:大量可调旋门，其中大多数组合未经测试，产生可疑的PPA效益。如果一个特性是RISC-V规范所要求的，但不是性能关键的，我们通过仿真来实现它。代码是“所有你需要的RTL，没有你不需要的。”
- 2)模块化。我们使用干净，延迟不敏感的接口，不依赖于其他模块内部的知识。这允许多个贡献者一起工作

彼此独立，并最大限度地减少由于对整个代码库的不完全理解而出现的错误。

- 3)友好。在设计决策和演示中，我们都会问自己:这样做会让项目更平易近人吗?有了这个，我们可以建立一个大型的开源项目文化，鼓励贡献，避免“不是我发明的”综合症。

系统架构

BlackParrot 多核实现了 RISC-V 64 位“RV64 G”架构，其中包括基数整数 ISA “I”，乘法和除法“M”，原子“A”，单精度和双精度浮点“F/d”。它支持三种特权级别——机器、管理器和用户，以及SV39虚拟内存;这些扩展足以有效地运行功能齐全的操作系统，如Linux。

无竞可编程缓存一致性

BlackParrot实现了一种基于分布式目录的缓存一致性协议，目前支持VI、MSI和MESI。底层实现BedRock由一组本地缓存引擎(lce)组成，每个lce控制一个L1缓存，它们通过互连网络连接到可编程缓存一致性引擎(CCEs)，这些引擎共同维护地址分片目录状态。

BedRock协议实现具有无竞争的独特属性，因为它们确保相干状态转换发生在CCEs而不是本地缓存或lce，这大大降低了协议的复杂性。

异构平铺多核SoC微体系结构

BlackParrot被设计成一个可扩展的、异构平铺的多核微架构，如图3(a)所示。(我们使用多核微架构这个术语，是因为用户或程序员并不知道多核的组件是如何安排的;它隐藏在多核ISA层之下。)将系统分解成可复制的子块有几个好处

:结构被正则化以实现可扩展性和易于定时关闭，系统可以灵活地组成不同的拓扑，协议复杂性可以从组件级别转移到网络级别。

Black-Parrot使用noc集合，而不是用共享总线连接这些块。BlackParrot使用的路由器是维度有序和虫洞交换的，使用基于信用的流量控制来限制拥塞。

为了提高层次化ASIC流的规律性，系统被设计成二维网格，每个网格中包含单个路由器。有些网络可能在一个网格中有多个端点——这些端点连接被组合起来，并通过虫洞集中器连接到路由器。虽然许多其他处理器使用基于标准总线的接口(如AXI或AHB)进行所有片上通信，但这些协议非常复杂，需要复杂的IP块来实现合理的性能。BlackParrot没有将其内部网络与任何特定的标准总线实现相耦合，而是提供了一组适配器来在一组众所周知的协议(如AXI或WishBone)之间进行转换。

BlackParrot微建筑瓷砖类型

BlackParrot微架构瓷砖属于图3(a)中列举的四类之一，我们将在下面详细介绍。

BlackParrot Core Tile包含一个完整的BlackParrot处理器或一个充当处理器的加速器，包括一个或多个连贯缓存以及一个目录分片和一个L2切片。一个典型的系统有许多Core tile。

L2扩展块提供了一种简单的横向扩展方法来增加BlackParrot系统中片上L2的数量。每个L2扩展包含一个目录和一个非包容非排他的L2切片。通过分配L2片，系统设计人员可以很容易地改变BlackParrot系统的计算与缓存比率，而不会干扰核心或noc内的关键路径。

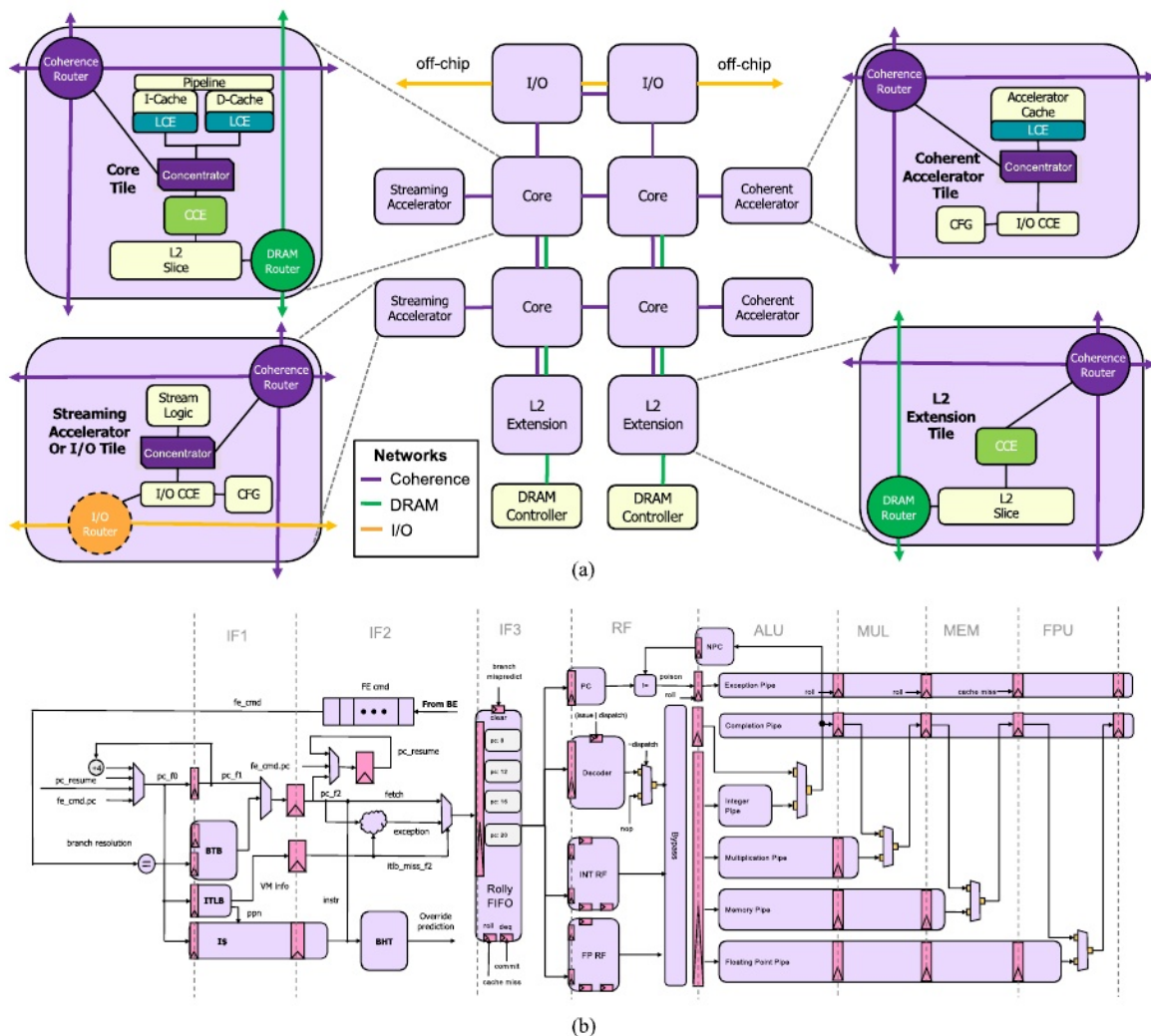


图3。(a) 黑鸚鵡多核soc由异构块组成，允许灵活地组合核心、加速器、L2缓存片、I/O和DRAM控制器。图中有四种不同的瓦片。核心磁片实现了一个处理器、一个目录碎片和一个L2片。相干加速器块实现了一个可以访问缓存相干存储器系统的加速器。L2扩展块允许改变L2缓存的数量。流媒体加速器或I/O块允许通过在相干网络上路由的共享非l1缓存地址空间灵活地连接公共内存系统。(b) 黑鸚鵡核心微架构。前端(图中IF1和IF2阶段)和后端都坚持接口规范，采用简单高效的流水线实现。由于适度的错误预测惩罚，复杂的分支预测器是不必要的。为了消除物理设计密集型的全局失速信号，后端在发布阶段之后不失速，而是在不频繁的缓存丢失时刷新管道并重播指令。缓存缺失完全通过BedRock相干系统处理。

将一个相干加速贴图附加到黑鸚鵡网络上，可以通过一些程度的特殊化来完成。从抽象的系统视图来看，相干加速器只是一个具有后备相干缓存的LCE。根据加速

器的需求和项目的复杂性预算，用户可能(按复杂性的递增顺序):

- 1)将加速器直接连接到所提供的黑鸚鵡数据缓存;

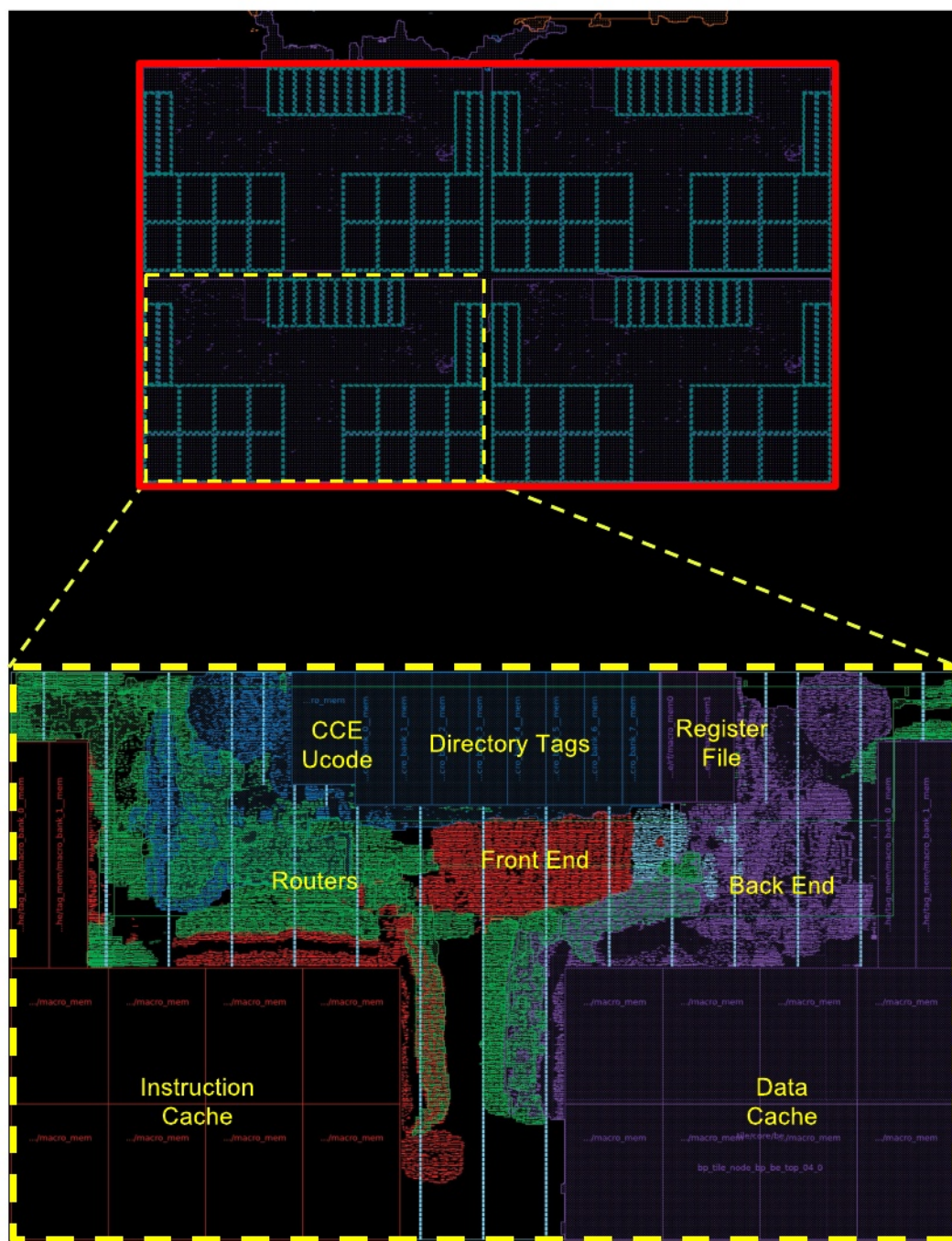


图4. 黑鸚鵡GF12放置并路由。2019年7月，采用GlobalFoundries 12纳米技术的四核黑鸚鵡系统问世。从这种胶布中吸取的经验教训推动了黑鸚鵡系统架构的几项重大变化，从而减少了50%的瓷砖面积，减少了50%的总电线长度。虽然rtl级仿真在分析和比较相对性能方面是有效的，但没有ASIC验证的实验可能会掩盖设计的严重物理限制。

- 2) 重用所提供的黑鸚鵡 LCE，并提供专用缓存；
- 3) 提供一个专门的LCE实现，直接与网络接口。

流媒体加速块流媒体块是没有本地缓存内存的块，并且在逻辑上不控制任何物理内存。也就是说，这些磁贴不包含备份

LCE的一致性缓存，而不是目录。这些块可以用于基本的I/O设备，网络接口链路，甚至重量级的基于流的加速器，如gpu。

其他Tile组件

作为一个真正的平铺微架构，BlackPar-rot分配了尽可能多的系统资源。每个黑鸚鵡内核块包含一个内存映射的配置块、一个内核级中断控制器切片和一个全局L2切片。分割这些系统资源促进了块中的规律性，消除了全局路由的配置和中断信号，最终简化了物理设计实现。通过对网络内存映射的简单调整，组件可以很容易地连接到黑鸚鵡系统，并从任何其他块进行寻址。

片上网络

在黑鸚鵡中有3个NoC类:Coherence (BedRock), DRAM和I/O。虽然noc是使用标准的Base-Jump STL模块实现的，但黑鸚鵡对协议的每个网络进行了专门设计，包括飞行宽度、数据包长度和坐标宽度，以优化物理设计。

BedRock网络BedRock网络是一个缓存一致的结构，连接黑鸚鵡系统中的所有块。具体来说，网络是系统中所有lce和ce的连接点。BedRock协议有三个逻辑通道:请求、命令和响应。请求通道由LCE用于启动事务，指定它是读还是写，是缓存还是不缓存，以及用于返回寻址的附加元数据。命令通道用于CCE修改系统的LCE状态。示例命令包括设置标签、填充数据和完成同步序列。此外，还可以命令lce在彼此之间传输缓存线——这些传输也通过命令网络进行。最后，响应通道用于一致性确认，允许请求的串行化。

虽然BedRock协议不需要它，但网络的当前实现是虫洞路由的二维网格，每个逻辑通道有一个物理通道。

DRAM网络DRAM网络将CCEs连接到能够服务内存请求的设备，例如DRAM、闪存或片上rom。由于所有请求都是由一个tile发起的，并由芯片底部的存储设备提供服务，因此内存网络是一个轻量级的0.5-D网络。DRAM网络特别适合虫洞路由，因为DRAM控制器倾向于首先返回最不重要的字。

IO网络用于将黑鸚鵡处理器连接到外设(如串口、PCIe控制器、外部I/O设备和调试接口)。消息可以在芯片上或芯片外初始化，因此I/O网络被实现为一维虫洞网络。这种网络只存在于I/O复合体中;一般来说，它充当黑鸚鵡协议和标准协议(如AXI、WishBone和简单的比特敲打)之间的轻量级传感器和物理传输层。

解耦的核心微架构

黑鸚鵡被设计成模块化的，获得了更简单的验证、更敏捷的开发和更容易的用户和开发人员入门的通常好处。此外，通过关注接口而不是具体的实现，黑鸚鵡可以支持各种可能的微体系结构。图3(b)显示了当前的黑鸚鵡核心微架构。

通过瘦接口实现效率

虽然软件接口抽象是一个有用的工具，但硬件接口有物理开销，这可能会削弱设计。黑鸚鵡中的接口被设计为具有最小的开销，划分逻辑上和物理上分离的区域。这里描述的每个接口都是作为一个可参数化的SystemVerilog结构体实现的，它通过一个延迟不敏感的端口传递，通常是通过一个小的FIFO。端解耦确保了这些逻辑分离的组件之间没有时序路径，并提供了一个端实现变化不会破坏另一个端的信心。

前端

前端向后端呈现一个有序但推测性的指令流。问题队列将前端提取与后端执行解耦，允许在长延迟后端操作(如服务缓存丢失)期间进行推测性提取。在指令提取期间，可能会出现异常。由于该领域的异常纯粹是推测性的，因此它们被发送到后端，与指令一起进行内联服务。因为RISC-V虚拟内存方案可能会在指令获取期间修改体系结构状态[设置“Access (A) bit”]，所以前端的所有TLB失误都被发送到后端，与其他异常一起内联处理。与PC/指令/异常对一起，前端还发送与导致特定PC获取的分支预测相关的元数据。

后端

后端执行指令，处理异常，通常维护处理器的体系结构状态。从后端到前端的消息用于纠正错误推测和更新前端的影子状态。消息包括分支解析、中断重定向、iTLB操作和特权模式更改。分支解析后，与分支关联的分支元数据被转发回前端。这个元数据永远不会被后端检查;前端使用的特定分支预测方案对后端完全不透明。

内存端

黑鸚鵡的内存端BedRock是一个可扩展的、分布式的、基于目录的一致性方案，其设计强调简单性和可验证性。相干系统中的节点要么是LCE，要么是CCE。CCEs负责管理物理地址空间片的一致性。lce负责代表一个相干缓存发起和响应相干请求。BedRock连接黑鸚鵡多核的所有组件，包括非相干或I/O设备。

Bedrock中有三种类型的CCE:一种新的微编程变体，一种传统的固定功能管理

引擎，以及只实现未缓存请求的最小控制器，用于I/O或简单的加速器。微编程CCE是黑鸚鵡核心的默认配置，在实现相干协议的变体时提供了极大的灵活性和适应性，但面积成本很小。值得注意的是，由于微代码可以通过简单的固件更新来更改，因此即使在现有的硅设计上也可以应用高级相干实验和安全补丁。

敏捷开发流程

在本节中，我们将描述黑鸚鵡开发工作的关键特性，这些特性使其成为用户可以信任的设计。

利用开源库

为了在黑鸚鵡上快速迭代，利用已建立的开源代码库是很重要的。通过构建经过实战测试的硬件库，如用于System-Verilog⁸和Berkeley Verilog HardFloat⁶的BaseJump STL，并集成使用延迟不敏感的设计原则，黑鸚鵡能够最大限度地减少其可能存在的错误。黑鸚鵡的开发是使用商业工具完成的，如Synopsys VCS，但至少支持一个开源替代方案，如Verilator⁷，是一个一流的基础设施问题。

评估设计复杂性(Design Complexity)和开箱即用体验

黑鸚鵡宣言中最难评估的部分之一是，我们是否实现了病毒式传播和复杂性的目标。为了评估这一点，我们在一个有35名四年级和五年级学生参加的VLSI课堂上，将BlackParrot作为一个为期三周的课堂项目。所有的学生(包括那些以前没有学过计算机体系结构的学生!)都成功地提出并实现了独特的、以前不支持的核心功能，比如修改管道，将负载使用延迟从三个周期改为两个周期，在L1缓存中添加可变填充宽度，在L1缓存中启用不同的银行大小，并在前端实现更复杂的分支预测器。我们打算每年进行一次

与黑鸚鵡相关的任务，以持续地自我评估项目的这些方面。

共同模拟测试框架

黑鸚鵡中的大部分测试是通过由程序级执行驱动的系统级测试台完成的。除了一些直接的白盒测试之外，黑鸚鵡还支持riscv测试套件、BEEBS套件、Spec和CoreMark作为基线功能回归。我们的计划是大大扩展这一选择。

黑鸚鵡采用并行协同仿真的混合方法，使用开源RISC-V ISA模拟器Dromajo。¹首先，使用Dromajo模拟一个长时间运行的程序。每N个周期，Dromajo收集系统的体系结构状态，并创建内存转储和检查点ROM，检查点ROM包含普通的RISC-V指令，旨在将刚重启的处理器初始化到定义良好的体系结构状态。接下来，并行地，使用每个检查点ROM恢复黑鸚鵡RTL模型，并与Dromajo模型共同模拟。中断和设备I/O等不精确的事件从RTL传递到Dromajo，以保持两个版本一致。

ASIC验证

除了我们正在实验室运行的12nm黑鸚鵡芯片外，还计划推出几款黑鸚鵡缘虫，既可以作为独立芯片，也可以作为加速器主机。包含tapeout参数化、约束和ASIC基础结构的目录都作为参考提供。通过UW OpenROAD Free45参考流程^{2,3}推动黑鸚鵡的工作正在进行中，这样用户就可以简单地克隆存储库并生成一个完全放置和路由的、具有代表性的黑鸚鵡。提供这种能力将使架构师能够快速验证他们的硬件实验，而无需维护商业CAD流程的财务和智力开销，或者需要签署限制性的代工nda。

结论

RISC-V挑战了x86和ARM的世界秩序。加州大学Berke-

我们欢迎您的光临。
增强!当黑鸚鵡变成
更广泛地使用，社区专家可以
回馈给黑鸚鵡，确保
它仍然是最先进处理器设计的代表。

莱伊带领了一个全球性的负责维护RISC-V ISA及其软件库。然而，尚未出现的是一个类似的linux支持的RISC-V实现的全球管理，它被文档化，PPA高效，并在标准SystemVerilog中实现。黑鸚鵡从底层开始设计，以填补这一角色，与

先前的efforts^{4,5,10}有集中的管理模型，并有机地发展到目前的状态。

黑鸚鵡小巧、模块化、友好。它是一个理想的SoC“基类”，可以与加速器集成并构建支持linux的系统。我们欢迎您的增强!随着黑鸚鵡得到更广泛的使用，社区专家可以为黑鸚鵡做出贡献，确保它仍然是最先进处理器设计的代表。BlackParrot现在可以在GitHub (<https://github.com-black-parrot/>)，使用BSD-3许可。

&参考文献

1. Dromajo. [Online]. Available: <https://github.com/chipsalliance/dromajo/>
2. UW openroad free45 PDK reference flow. [Online]. Available: https://github.com/bsg-idea/uw_openroad-free45/
3. Ajayi et al., “Toward an open-source digital flow: First learnings from the openroad project,” in Proc. 56th Annu. Design Autom. Conf., 2019, Art. no. 76.
4. Asanovic et al., “The rocket chip generator,” UC Berkeley EECS Tech. Rep. UCB/EECS-2016-17.
5. Celio et al., “The Berkeley out-of-order machine (boom): An industry-competitive, synthesizable, parameterized RISC-V processor,” Electr. Eng. Comput. Sci. Dept., Univ. California Berkeley, Tech. Rep. UCB/EECS-2015-167.
6. J. Hauser, Berkeley HardFloat. [Online]. Available: <http://www.jhauser.us/arithmetic/HardFloat.html>
7. W. Snyder, “Verilator: Fast, free, but for me?” DVClub Presentation, 2010, p. 11.

8. M. Taylor, “Basejump STL: SystemVerilog needs a standard template library for hardware design,” in Proc. 55th Annu. Design Autom. Conf., 2018, Art. no. 73.
9. A. Waterman et al., “The RISC-V instruction set manual, volume i: Base user-level isa,” EECS Department, UC Berkeley, Tech. Rep. UCB/EECS-2011-62 116, 2011.
10. F. Zaruba and L. Benini, “The cost of application-class processing: Energy and performance analysis of a linux-ready 1.7-GHz 64-bit RISC-V core in 22-nm FDSOI technology,” IEEE Trans. Very Large Scale Integr. (VLSI) Syst., vol. 27, no. 11, pp. 2629–2640, Nov. 2019.

Daniel Petrisko是西雅图华盛顿大学的博士生。联系他的邮箱是petrisko@cs.washington.edu。

法扎姆·吉拉尼是西雅图华盛顿大学的博士生。联系他的邮箱是farzamgl@uw.edu。

Mark Wyse是西雅图华盛顿大学的博士生。联系他的邮箱是wysem@cs.washington.edu。

戴哲(Dai Cheol Jung)是西雅图华盛顿大学的博士生。联系他的邮箱:dcjung@uw.edu。

斯科特·戴维森(Scott Davidson)是西雅图华盛顿大学的博士生。联系他的邮箱是stdavids@uw.edu。

保罗·高(Paul Gao)是西雅图华盛顿大学的博士生。联系他:gaozihou@uw.edu。

赵春, 西雅图华盛顿大学博士后研究员。联系邮箱: chunzhao@uw.edu。

扎赫拉·阿扎德是波士顿大学的博士生。联系她的邮箱是zazad@bu.edu。

Sadullah Canakci是波士顿大学的博士生。联系他的邮箱是scanakci@bu.edu。

Bandhav Veluri目前正在西雅图的华盛顿大学攻读硕士学位。联系他的邮箱是bandhav@uw.edu。

塔维奥·瓜里诺目前正在攻读西雅图华盛顿大学的硕士学位。联系他的邮箱是Tavio.guarino@gmail.com。

Ajay Joshi目前是波士顿大学的教授。联系他:joshi@bu.edu。

马克·奥斯卡目前是西雅图华盛顿大学的教授。联系他的邮箱是oskin@cs.washington.edu。

迈克尔·贝德福德·泰勒目前是西雅图华盛顿大学的教授。联系他的邮箱是prof.taylor@gmail.com。

面向边缘计算设备的可靠RISC-V内核

Pegdwende Romaric Nikiema¹, Alessandro Palumbo², Allan Aasma³, Luca Cassano², Angeliki Kritikakou^{1,5},
Ari Kulmala³, Jari Lukkarila³, Marco Ottavi⁴, Rafail Psiakis³, Marcello
Traiola¹¹法国雷恩大学, Inria, IRISA, CNRS, ²意大利米兰理工大
学,

³阿拉伯联合酋长国技术创新研究所,

⁴荷兰特温特大学和意大利罗马大学⁵法国大学学院(IUF) –

1{pegdwende.nikiema;angeliki.kritikakou;marcello.traiola}@inria.fr, ²first name。最后name@polimi.it,
³名。最后一个name@tii.ae, ⁴m.ottavi@utwente.nl

摘要:将计算从云迁移到边缘设备, 即物联网(iot)设备, 可以减少延迟和流入网络的数据量。随着新兴的开源和可定制的RISC-V指令集架构(ISA), 基于这种ISA的内核是物联网家族中几个应用领域的有希望的候选者, 例如汽车, 未命名飞行器(无人机), 工业自动化, 医疗保健, 农业等, 其中功耗, 实时执行, 安全性和可靠性是最重要的。在这个新兴的连接RISC-V物联网设备的新时代, 需要可靠和安全的执行机制, 仍然满足边缘设备的面积, 能耗和计算时间限制。为了实现这一目标, 我们提出了三种机制, 即(i)用于后量子安全引导的信任根模块, (ii)针对硬件特洛伊木马和微架构侧信道攻击的硬件检查器, 以及(iii)用于实时错误检测和纠正的细粒度双核锁步机制。本文阐述了具有相关动机和含义的拟议机制, 并对未来的研究方向进行了讨论。

索引术语:risc - v, OpenTitan, RoT, 安全启动、后量子密码术、PQC、安全、安全、晶体二锂、同步双核、恢复、安全检查器、硬件特洛伊木马、微架构侧通道攻击。

I. 引言

为了不仅减少延迟, 而且减少流入网络的数据量, 有一种动机将计算从云迁移到边缘设备, 即物联网(iot)设备。随着开源、可定制的RISC-V指令集架构(Instruction Set Architecture, ISA)的兴起, RISC-V进入了前台, 越来越受到业界的关注。RISC-V是物联网家族中几个应用领域的有前途的候选者, 例如汽车, 未命名飞行器(无人机), 工业自动化, 医疗保健, 农业等, 其中功耗, 实时执行, 安全性和可靠性至关重要[2], [3]。然而, 大多数RISC-V工作主要集中在性能和功耗设计上

消耗, 往往忽略了可靠性和安全性问题[4]。在这个新兴的连接RISC-V物联网设备的新时代, 需要可靠和安全的执行机制, 满足边缘设备的面积, 能耗和计算时间限制。

为了实现这一目标, 这样的机制应该解决完整的系统堆栈。构建整个可靠和安全堆栈的基本机制是安全引导, 即对执行的代码进行身份验证并验证其完整性。在系统启动之后, 必须保护系统执行免受恶意或无意注入的错误。这样的错误会影响指令和数据从存储器到处理器的提取, 甚至影响处理器的执行。

在这项工作中, 我们提出了三种应用于系统堆栈不同层的机制, 以实现可靠和安全的RISC-V内核。作为第一步, 应该通过安全启动方案建立强大的可靠性基础, 使用信任根(RoT)模块(如OpenTitan)绑定到硬件, 该模块使用基于公钥密码术(PKC)算法(如RSA和ECC)的数字签名。然而, 量子计算技术的出现使得整数分解和基于离散对数的密码学, 如RSA和ECC, 分别变得容易被破解。因此, 需要适合后量子时代的数据、代码完整性和认证解决方案。然而, 由于其资源限制, OpenTitan无法在其安全引导流中支持现有的公钥后量子加密(PQC)算法, 而无需进行重大修改。为了解决这一限制, 我们提出了OpenTitan RoT的增强安全引导流程, 修改OpenTitan硬件和软件架构, 实现后量子安全。

第二步, 为了防止在数据从内存传输到处理器的过程中受到攻击, 提出了防止执行不需要的软件和访问非法内存位置的机制。为了实现这一目标, 我们提出了硬件机制来监控抓取活动、处理的数据和micropro的状态

处理器,以检测潜在的可疑活动,即硬件特洛伊木马的激活或运行微架构侧信道攻击的尝试。

最后,为了处理处理器内部发生的不同性质的硬件故障(即,与制造缺陷、老化或辐射诱发有关),通常使用基于冗余的机制:在不同的处理器上执行具有相同输入的不同操作集。这种方法非常有效,因为在所有处理器上同时发生相同故障的概率非常小[7]。然而,当它应用于粗粒度级别时,它会带来很高的错误检测和纠正成本。因此,我们提出了一种具有快速错误检测和纠错功能的细粒度锁步版本RISC-V处理器。我们采用高级合成(High-Level Synthesis, HLS)设计了该方法。与HDL实现相比, HLS使设计过程不那么复杂,因为处理器模型可以很容易地修改、扩展和验证。

本文的提示内容组织如下:第二节介绍了后量子安全启动机制;第三节介绍了反木马和反侧信道攻击的硬件安全检查器;第四节讨论了用于错误检测和纠错的双核锁步实现;最后,第五节对论文进行总结。

II. opentitan rot上的后量子安全启动

A. 动机

OpenTitan是一个开源的基于RISC-V的信任根(RoT),其中一个用例是通过安全启动过程建立安全信任链的基础。如前所述,为了安全启动, OpenTitan目前使用的PKC方案不是后量子安全的,因此我们的目标是确定OpenTitan的最佳候选算法。

目前的PQC数学解决方案是基于哈希、编码、多变量和格子[9]。考虑到RISC-V边缘设备的资源约束环境, PQC安全启动算法需要满足面积和能耗以及计算时间的要求。提出了使用后量子哈希签名方案的第一个基于PQC硬件的安全引导实现,并与硬件和软件ECDSA安全引导解决方案[10]进行了比较。基于LMS和SPHINCS+算法的高效PQC UEFI安全引导的PQC安全引导签名具有最小的身份验证时间和引导时间,以及内存占用[11],使这些算法非常适合资源有限的边缘设备。

[12]中提出了与传统公钥方案相比,在受限嵌入式设备上基于格子的加密实现的调查。结果表明,随着密钥大小的增加,基于格的密码方案的性能会恶化。在ARM cortex-M4微处理器上测量不同晶格方案的性能时, crystals - diliium方案具有最高的吞吐量性能。crystals - diliium也是NIST PQC标准化项目[13]第三轮的两个标志性入围者之一

,使其成为OpenTitan中实施的良好候选者。然而,这种安全方案意味着复杂的软件实现,并且由于边缘设备的ROM大小限制,在物联网设备的第一启动阶段引入它并不简单。

为了克服这一限制并提供抗PQC启动方案,我们扩展了OpenTitan的RTL以支持crystals - diliium加速。crystals - diliium硬件原始[14]在我们的设计中使用,以验证引导的下一阶段的签名。

B. 针对OpenTitan提出的PQC安全引导

在OpenTitan上提出的PQC安全引导确保在系统启动时只执行已验证的有效负载。请注意,本文中讨论的安全引导流程不包括BL0固件的后期引导阶段和所支持的操作系统内核引导。引导过程的第一阶段位于引导ROM中,ROM代码在制造过程中被编程到芯片的ROM中,并且它是不可变的。它包含一组用于验证存储在闪存中的第一引导阶段(即ROM EXT)的公钥。

ROM代码的主要任务是为执行ROM EXT准备opentitan,同时确保加载的ROM EXT允许在该芯片上执行(ROM扩展-存储在闪存中并由Silicon Creator签名)。图1给出了启动代码的两个步骤的概述。一旦系统开机/复位,启动过程

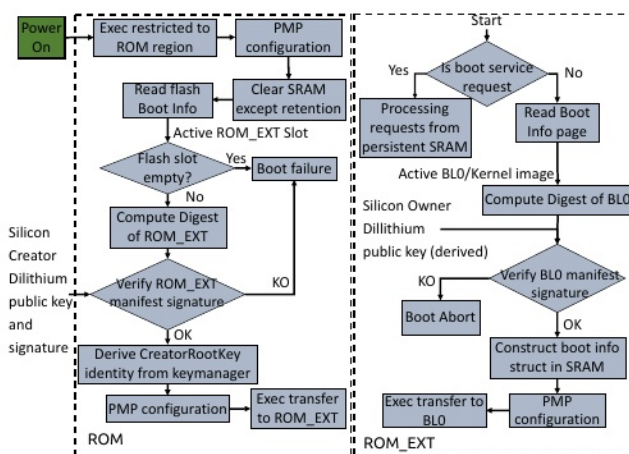


图1. OpenTitan PQC安全启动代码。

启动。执行仅限于位于OpenTitan内部的OnChip ROM。其他类型/内存区域的执行最初被重置逻辑阻止。执行进入ROM代码,启用并配置增强物理内存保护(OpenTitan的Ibex核心的ePMP)。在这个阶段,内核只能访问可执行引导代码所在的ROM区域。清除除保留SRAM外的所有SRAM。主引导槽从flash boot Info加载。Flash引导信息驻留在Flash默认银行中。从活动ROM EXT插槽开始,ROM代码执行以下步骤:

- 它确定是否Active ROM EXT插槽是空的，通过测试头魔术值的存在。如果它存在，它继续ROM EXT的验证，如果它不存在，它执行引导失败逻辑。-
- 如果Active ROM EXT槽位为空，则引导过程读取ROM EXT代码并计算其SHA2-256摘要。
- 使用CRYSTALS-Dillithium硬件原语，我们使用摘要和存储在ROM中的Silicon Creator公钥对ROM EXT清单中的签名进行验证，如果验证成功，则继续该过程。如果验证失败，该进程执行启动失败逻辑。
- 之后，它执行系统状态测量，并派生出creatorroot key标识，它将驻留在密钥管理器中，作为中间状态。
- 接下来，它通过配置适当的PMP条目来启用ROM EXT的执行。-
- 最后，它将执行转移到ROM EXT清单中为活动插槽指定的入口点。
- 执行跳转到flash中，即eXecute In Place (XIP)，并进入ROM EXT代码部分。-
- ROM EXT读取引导信息页，以确定应该启动哪个BL0/内核映像。
- ROM EXT读取BL0并计算其SHA2-256摘要。
- 使用CRYSTALS-Dillithium硬件原语，我们使用摘要和从密钥管理器(KM)派生函数派生的硅所有者公钥，在BL0 manifest中执行签名验证。如果验证成功，该过程继续进行。如果验证失败，则进程执行启动失败逻辑。

执行被转移到硅所有者代码的入口点(从BL0/内核映像清单)。执行进入硅所有者代码(即BL0，内核应用程序和数据，这些都超出了本文的范围)。

表一
晶体- dillithium FPGA在opentitan中的实现

Design	LUTs	Relative Footprint	Clk(MHz)
OpenTitan	166.180	-	10
Dilithium	55.123	33%	2.5

我们将提出的方法集成到原始的Open-Titan管道上，并在Xilinx Virtex UltraScale+ VCU118 FPGA板上进行了评估。FPGA实现结果如表1所示。提议的Dillithium硬件原语占整个OpenTitan设计的33%，因为该实现针对高性能进行了调优。为了避免同步问题，我们不得不将其时钟降低到OpenTitan频率的1/4。为了降低上述成本，作为未来的工作，我们将在我们的下一个硬件原语设计中权衡性能以获得面积占用增益，并实现硬件加速器解决方案以卸载Dilithium算法的点阵计算。在硬件中只卸载

算法中最苛刻的部分，即随机生成和多项式乘法运算，可以与纯硬件实现进行研究和比较。

III. 硬件安全检查器

A. 动机

RISC-V架构本身具有许多安全扩展，如特权级别、物理内存保护和加密[15]。然而，这些本地机制并不能保护微处理器免受两种日益上升的威胁:硬件特洛伊木马和微架构侧信道攻击[16]。

长期以来，硬件特洛伊木马(HTHs)一直被认为是一个纯粹的学术问题，因为它们通常暴露出有限的复杂性和危险性。然而，最近出现了一个新的威胁，这使得HTH也引起了工业界的关注:软件可利用HTH[17]。复杂和高度危险的HTHs可能会感染实现微处理器的IP内核:这种HTHs可能允许攻击者执行恶意软件，修改正在运行的软件，获得未经授权的特权或窃取机密信息。最近，在Via Technologies公司的商用C3处理器[19]中发现了一个真实世界的HTH，即Rosenbridge后门。这个HTH可以通过软件激活和利用，进入系统的管理员模式¹。虽然已经提出了几种电路级设计时HTH检测技术[20]-[22]，但人们对系统级技术的兴趣越来越大，这些技术允许使用不可信组件[23]，[24]构建可信系统，或者在(部分)不可信系统[25]，[26]上执行可信软件。我们认为，现有的解决方案没有考虑到那些通过使CPU运行正常(但意外)指令而不改变特权模式而改变系统功能的HTHs。换句话说，这些工作都不会检查微处理器是否正在执行不需要的软件，以及它是否正在访问非法的内存位置。

最近几年出现的另一个威胁是微架构侧信道攻击(msca)[27]，例如Spectre和Meltdown。这些攻击允许窃取未经授权的信息，而不需要攻击者对被攻击的系统进行物理访问。事实上，此类攻击仅依赖于利用合法的HW特征，例如推测执行和分支预测，以及在运行敏感应用程序时观察系统的定时行为。在过去的几年里，人们已经提出了一些针对msca的对策。恒定时间技术，依靠使执行时间恒定来保护机密信息;已经提出了编译时技术来识别和修改那些可能为msca打开大门的指令序列;并提出了基于缓存分区和周期性缓存刷新的操作系统级解决方案。所有这些技术都存在适用性有限和速度明显减慢的问题。最后，一个很大的

¹在[19]发布后，Via Technologies官方评论说，这种行为是由于一个用于调试的未记录的特性。

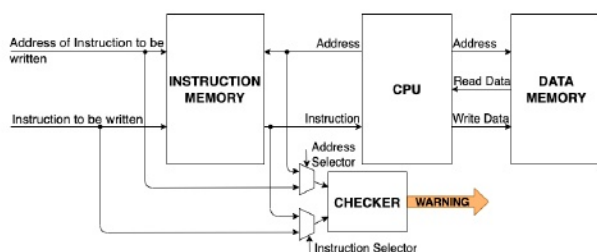


图2. 安全架构。

存在许多利用机器学习和观察硬件性能计数器的技术。这些技术实现了非常高的检测精度，但它们都需要多线程或专门用于执行检测引擎的额外(可信且不受攻击的核心)。因此，这些技术可以应用于高性能计算系统，例如高端服务器，但不适用于边缘设备和低端系统，例如智能卡或汽车嵌入式系统，其中单核可用，操作系统要么不可用，要么极其简单。

B. The Proposed Hardware Security Solutions

我们的解决方案(其高级表示如图2所示)旨在允许系统集成商通过购买可能不可信的组件来构建可信的系统。因此，我们认为IP提供者是攻击者;另一方面，系统集成员工和代工被认为是可信的。我们在架构中引入了硬件安全检查器(hsc)，这样hsc就能够监控抓取活动、处理的数据和微处理器的状态，以便检测潜在的可疑活动，即HTHs和/或msca激活。这些hsc分两个阶段工作:在第一个设计时配置步骤中，它们学习²哪些指令/指令序列是合法的。在随后的运行时监控步骤中，它们检查微处理器的活动是否与先前学习的合法配置文件相对应。

1)硬件木马检测:在b[29]中，我们提出了一种基于Bloom滤波器的HSC，以保护微处理器免受主存HTHs的侵害。HSC是在软件安装到指令存储器时通过存储有关合法指令和相应内存地址的信息来配置的。在运行时，HSC监视由微处理器访问的内存位置和相应的获取指令，并基于此信息查询Bloom过滤器。如果访问的内存地址和/或相应的获取指令不合法，HSC将发出警报。一个较轻的HSC版本已于2010年提出。

我们在PULPINO平台的RI5CY版本中集成了建议的反hth安全检查器，这是一个小型的

²虽然提出的解决方案不依赖于机器学习/人工智能，但这里使用了“学习”一词。

4级RISC-V内核。在Xilinx Artix XC7A35T上合成时，RI5CY需要15097个lut和9881个ff，工作频率约为50MHz，总功耗为127mW。我们考虑了以下基准:二分搜索(bin)、矩阵乘法(MM)、冒泡排序(BubS)、快速排序(QS)、数数解算器(SS)和运动检测(MD)。我们模拟了10,000个案例，其中HTH强制从意外的内存地址进行恶意读取，以及10,000个案例，其中HTH修改从合法地址读取的获取指令。表II报告了两种攻击场景的假阳性(假警报)和假阴性(未检测到的攻击)率。可以观察到，我们总是有0%的误报率，平均只有约2%的攻击未被检测到，但只有在HTH修改获取的指令时。在开销方面，我们引入了大约10%的面积增加，大约2%的功耗增加，并且没有降低工作频率。

表2 HTH检测精度

Bench.	Address modification		Instruction modification	
	FP	FN	FP	FN
BinS	0%	0%	0%	2.25%
MM	0%	0%	0%	0.40%
BubS	0%	0%	0%	3.01%
QS	0%	0%	0%	3.91%
SD	0%	0%	0%	0.72%
MD	0%	0%	0%	2.83%
AVG	0%	0%	0%	2.18%

2)检测微架构侧信道攻击:b[31]中提出了一种类似于先前提出的方法，但用于识别msca的激活。我们利用Count-Min Sketch模型在运行时检测代表特定msca的预定义和可重复的可疑指令模式集的执行情况。安全工程师在设计时仔细识别这些指令模式，然后用于配置HSC，每种攻击都对应一个指令模式。因此，我们的建议能够并行检查几个微架构攻击，从而代表了一个灵活和可扩展的安全解决方案。

我们将HSC集成到RSD核心中，RSD核心是一个32位、无序、超标量、两取前端和五个问题后端管道的RISC-V核心，具有16 KByte指令缓存。我们在Virtex7 FPGA上实现了目标微处理器，采用18334个lut，10885个ff，4512个LUTRAM单元和17个BRAM单元，工作在57 MHz，估计功耗约为0.926W。我们认为Coremark、Towers、RSort和Median是受到攻击的良性程序，而Spectre、Orchestration、Flush+Reload和Rowhammer是msca程序。我们对每个良性程序都进行了10000次实验-攻击程序，结果是100%的攻击被检测到。然后，我们测量了引入的HSC引起的假警报的数量:范围从

根据HSC本身的结构，从80%到0%不等。最小的HSC配置允许我们获得0%的假阳性和假阴性率，但却带来了大约10%的面积开销，大约4%的功耗增加，并且没有降低工作频率。

IV. 锁步双核

A. 动机

处理器冗余可以以非侵入式或侵入式的方式实现，使用同构或异构处理器。非侵入式方法在核心层引入冗余，而不修改内部处理器微架构，因此通常在内部微架构细节不可用或难以修改时使用，例如商用现货(COTS)处理器。例如，使用两个相同的MicroBlaze软核来实现同质双核LockStep (DCLS)[32]，其中故障处理器被排除在计算之外，并通过重新配置进行修复，而正确的处理器继续工作。异构非侵入式DCLS使用RISC-V软核和ARM A9硬核[33]。在应用程序中使用检查点来检查内核之间的不匹配，并在检测时使用回滚策略。异构方法可能会降低性能，低速处理器是DCLS性能的上限。此外，与硬核同步执行需要特定的架构支持，这在所有处理器[33]上都不存在。总体而言，非侵入式方法易于实现，因为不需要对微架构进行修改。然而，它们将定义纠正策略的责任留给了应用程序级别(例如，通过常规检查点)，这可能导致较高的时间开销。

另一方面，侵入式方法修改了内部处理器微体系结构，提高了实现纠正机制(例如回滚)时的灵活性。例如，动态自适应冗余架构(Dynamic Adaptive Redundancy Architecture, DARA)是一种应用于RISC ISA SH-2处理器的同质侵入式DCLS方法，通过回滚[34]实现纠错。DARA增加了额外的硬件来检查锁步内核之间所有管道级寄存器的一致性。然而，DARA不支持发生在分支指令中的错误。另一种同质侵入式方法使用两个虚拟RISC-V内核，通过细粒度交错多任务来实现DCLS，以解决共模故障(CMFs)[35]。其他方法通过错误检测和纠错代码扩展流水线寄存器，例如，Duckcore在流水线寄存器[36]中扩展了具有单错误校正双错误检测(SECDED)的RISC-V内核。然而，由于编码和解码时间的原因，这种方法可能会增加显著的开销，并且不能保证对管道阶段的计算逻辑中的故障进行保护。其他方法是在RISC-V核心内重复组件以提高其可靠性。例如，控制寄存器和状态寄存器，程序计数器和寄存器文件[37]，ff, lut, BRAMS和ldsp[38]，以及

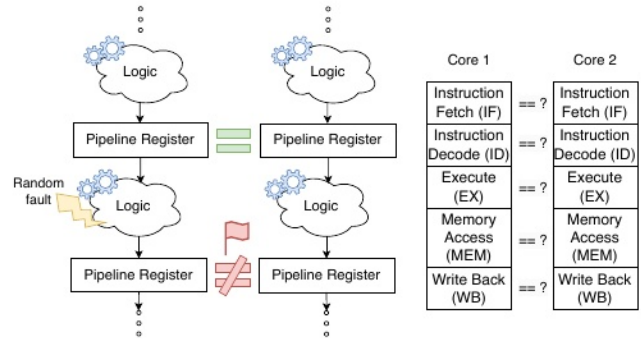


图3. 双核锁步原理说明

算术和逻辑单元(ALU)是三倍的[39]。总体而言，侵入式方法为实现有效的故障纠正机制提供了高度的灵活性，但实现起来可能具有挑战性。事实上，现有的侵入性方法是基于HDL实现的，通常是相当复杂的。

最后，上述描述的方法并不专注于提供有界的错误检测和纠正时间。然而，当嵌入式系统需要硬实时和可靠执行的领域中使用，这是关键，其中应用程序的最坏情况执行时间(WCET)也必须在可能存在故障的情况下受到限制。

B. 建议的方法

为了解决上述问题，我们提出了一种基于risc - v的侵入式同构双核锁步进实现，通过有限数量的额外时钟周期提供瞬态错误检测和纠正。

特别是，在2010年，我们采用了开源的Comet RV32I RISC-V架构，通过高级合成(HLS)实现了两种侵入式的锁步设计，即带回滚的部分阴影寄存器(PSRR)和完全阴影寄存器(FSR)。我们使用两个相同的RISC-V内核在每个时钟周期执行相同的指令。每个流水线阶段将其逻辑计算的结果存储在流水线寄存器中。在每个周期，所提出的机制通过比较两个核心的管道寄存器来检查执行一致性，如图3所示。如果没有检测到错误，则执行正常运行。否则，检测到的错误表明故障影响了逻辑，从而产生了错误的结果，或者故障影响了管道寄存器本身，在结果中翻转了一个位。在这种情况下，立即进行校正。在PSRR方法中，错误的指令被重新获取并再次通过整个管道，类似于DARA[34]。先前仍在管道中的指令不受影响，因为它们的执行不受错误的影响；但是，已经在管道中的后续指令将被丢弃。在FSR方法中，当没有检测到故障时，我们创建所有管道寄存器的备份副本。然后，在检测到故障时，丢弃当前计算的结果，恢复两个内核的管道寄存器

³ <https://gitlab.inria.fr/srokicki/Comet/-/tree/master>

使用备份值,最后两个内核重新执行受故障影响的周期。

此外,在[41]中,我们揭示了以下关键方面:影响核心的瞬态故障不仅会影响应用程序的功能行为,而且还会对其定时行为产生重大影响,从而影响WCET估计。为了实现这一目标,我们利用典型的无故障WCET估计来实现故障感知,通过考虑发生在核心上的瞬态故障的影响。更准确地说,我们首先通过广泛的故障注入对目标系统进行漏洞分析。该分析不仅验证了功能正确性,还验证了应用程序在核心上执行时的时序正确性。然后,应用一种典型的基于度量的WCET估计方法来验证故障对WCET估计的影响。

我们在TACLeBench(二分搜索和素数)、MiBench(Qsort)、AxBench(移动平均)和Polybench(矩阵乘法)的基准测试上进行了实验。从获得的结果中,我们观察到,与没有故障的应用程序执行时间相比,在存在瞬态故障的情况下,应用程序的执行时间可以显著增加,最高可达700%。此外,与无故障情况下的分布相比,执行时间轨迹的分布也得到了显著的修改。上述观察结果有直接的后果;在故障情况下完成执行所需的时间可能明显高于无故障WCET。因此,现有的方法应该使用看门狗计时器,以约束瞬态故障对应用程序执行时间的影响,并保证整体调度的安全。当计时器到期或检测到错误时,根据不同的方法,应用程序需要全部或部分地重新执行,从而导致较高的错误检测和纠正时序开销。我们证明了所提出的DCLS方法在错误发生时立即纠正错误-在传播并影响执行时间之前。因此,该方法提供了有限的定时开销(即,在FSR情况下,两个时钟周期)并恢复WCET估计。

V. 讨论和结论

基于RISC-V架构的边缘设备正在兴起,导致对安全可靠系统的需求。目前的工作提出了部署在RISC-V设备上的三种机制来实现这一目标。由于签名验证算法在后量子时代被证明是不安全的,我们使用OpenTitan RoT的物联网设备提出了一种后量子安全的安全启动机制。此外,为了检测潜在的可疑活动,我们展示了监控抓取活动的硬件机制,处理过的数据和微处理器的状态。最后,在双核锁步侵入式RISC-V上展示了两种具有有限WCET开销的机制,用于检测和恢复辐射诱发的瞬态故障。

此外,我们想强调RISC-V开放ISA可以为可靠性研究社区带来的优势。大量可用的开源核心实现和

围绕RISC-V的大型社区使研究人员能够轻松实现和测试他们的解决方案,以解决不同的可靠性问题。然而,我们注意到,并没有很多方法试图用统一的方法来解决不同的挑战,例如,使用相同的硬件来提供容错和保密性[42]。相反,不同领域(例如,可靠性和安全性)的专家通常独立决定他们感兴趣的需求,通常具有有限的交互。此外,非功能性需求通常被视为可选商品,这增加了相关解决方案的成本。我们认为,具有不同专业知识的不同群体的研究人员应该联合起来,并将可靠性要求作为未来主流应用中使用的RISC-V内核的主要设计要求之一。通过这种方式,确保高效可靠操作的统一高度优化实现可以蓬勃发展,并为未来边缘计算设备的RISC-V内核提供动力。

致谢

本研究的部分工作由法国国家研究机构(ANR)通过FASY研究项目(ANR-21-ce25-0008)资助。部分工作得到了阿联酋阿布扎比技术创新研究所的资助。

参考文献

- [1] C. Palmiero *et al.*, “Design and implementation of a dynamic information flow tracking architecture to secure a risc-v core for iot applications,” in *2018 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, 2018, pp. 1–7.
- [2] J. Abella *et al.*, “Security, reliability and test aspects of the risc-v ecosystem,” in *2021 IEEE European Test Symposium (ETS)*, 2021, pp. 1–10.
- [3] J. Anders *et al.*, “A survey of recent developments in testability, safety and security of risc-v processors,” in *2023 28th IEEE European Test Symposium (ETS)*, 2023.
- [4] A. D’orflinger *et al.*, “A comparative survey of open-source application-class risc-v processor implementations,” in *Proceedings of the 18th ACM International Conference on Computing Frontiers*, ser. CF ’21. New York, NY, USA: Association for Computing Machinery, 2021, p. 12–20. [Online]. Available: <https://doi.org/10.1145/3457388.3458657>
- [5] Y. H. Hwang, “Iot security & privacy: threats and challenges,” in *Proceedings of the 1st ACM workshop on IoT privacy, trust, and security*, 2015, pp. 1–1.
- [6] P. W. Shor, “Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer,” *SIAM review*, vol. 41, no. 2, pp. 303–332, 1999.
- [7] M. Cui *et al.*, “Fault-tolerant mapping of real-time parallel applications under multiple dvfs schemes,” in *IEEE RTAS*, 2021, pp. 387–399.
- [8] S. Rokicki *et al.*, “What you simulate is what you synthesize: Designing a processor core from c++ specifications,” in *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2019, pp. 1–8.
- [9] V. Mavroeidis *et al.*, “The impact of quantum computing on present cryptography,” *arXiv preprint arXiv:1804.00200*, 2018.
- [10] V. B. Kumar *et al.*, “Post-quantum secure boot,” in *2020 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2020, pp. 1582–1585.
- [11] P. Kampanakis *et al.*, “Post-quantum lms and sphincs+ hash-based signatures for uefi secure boot,” *Cryptology ePrint Archive*, 2021.
- [12] A. Khalid *et al.*, “Lattice-based cryptography for iot in a quantum world: Are we ready?” in *2019 IEEE 8th international workshop on advances in sensors and interfaces (IWASI)*. IEEE, 2019, pp. 194–199.

- [13] J. Zheng *et al.*, "Parallel small polynomial multiplication for dilithium: A faster design and implementation," in *Proceedings of the 38th Annual Computer Security Applications Conference*, ser. ACSAC '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 304–317. [Online]. Available: <https://doi.org/10.1145/3564625.3564629>
- [14] L. Beckwith *et al.*, "High-performance hardware implementation of crystals-dilithium," in *2021 International Conference on Field-Programmable Technology (ICFPT)*. IEEE, 2021, pp. 1–10.
- [15] A. Waterman *et al.*, "The risc-v instruction set manual volume ii: Privileged architecture document version 20190608-priv-msu-ratified," RISC-V Foundation, Tech. Rep., 2019.
- [16] L. Cassano *et al.*, "Is risc-v ready for space? a security perspective," in *2022 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFT)*, 2022, pp. 1–6.
- [17] Y. Wang *et al.*, "Software exploitable hardware trojans in embedded processor," in *2012 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFT)*, 2012, pp. 55–58.
- [18] Y. Jin *et al.*, "Exposing vulnerabilities of untrusted computing plat-forms," in *Proc. Int. Conf. Computer Design*, 2012, pp. 131–134.
- [19] C. Domas, "Hardware backdoors in x86 cpus," <https://i.blackhat.com/us-18/Thu-August-9/us-18-Domas-God-Mode-Unlocked-Hardware-Backdoors-In-x86-CPUs-wp.pdf>, 2018.
- [20] X. Chuan *et al.*, "An efficient triggering method of hardware Trojan in AES cryptographic circuit," in *Proc. Int. Conf. Integrated Circuits and Microsystems*, 2017, pp. 91–95.
- [21] Y. Liu *et al.*, "Scca: Side-channel correlation analysis for detecting hardware trojan," in *Proc. Int. Conf. Anti-counterfeiting, Security, and Identification*, 2017, pp. 196–200.
- [22] A. Palumbo *et al.*, "Is your fpga bitstream hardware trojan-free? machine learning can provide an answer," *Journal of Systems Architecture*, vol. 128, p. 102543, 2022.
- [23] J. Dubeuf *et al.*, "Run-time detection of hardware trojans: The processor protection unit," in *2013 18th IEEE European Test Symposium (ETS)*, 2013, pp. 1–6.
- [24] G. Bloom *et al.*, "Os support for detecting trojan circuit attacks," in *2009 IEEE International Workshop on Hardware-Oriented Security and Trust*, 2009, pp. 100–103.
- [25] L. Cassano *et al.*, "Deton: Defeating hardware trojan horses in microprocessors through software obfuscation," *Journal of Systems Architecture*, vol. 129, p. 102592, 2022.
- [26] —, "On the optimization of software obfuscation against hardware trojans in microprocessors," in *2022 25th International Symposium on Design and Diagnostics of Electronic Circuits and Systems (DDECS)*, 2022, pp. 172–177.
- [27] A. P. Fournaris *et al.*, "Exploiting hardware vulnerabilities to attack embedded system devices: a survey of potent microarchitectural attacks," *Electronics*, vol. 6, no. 3, p. 52, 2017.
- [28] Y. Lyu *et al.*, "A survey of side-channel attacks on caches and counter-measures," *Journal of Hardware and Systems Security*, vol. 2, no. 1, pp. 33–50, 2018.
- [29] A. Bolat *et al.*, "A microprocessor protection architecture against hardware trojans in memories," in *2020 15th Design & Technology of Integrated Systems in Nanoscale Era (DTIS)*. IEEE, 2020, pp. 1–6.
- [30] A. Palumbo *et al.*, "A lightweight security checking module to protect microprocessors against hardware trojan horses," in *2021 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFT)*. IEEE, 2021, pp. 1–6.
- [31] K. ArXkan *et al.*, "Processor security: Detecting microarchitectural attacks via count-min sketches," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 30, no. 7, pp. 938–951, 2022.
- [32] A. Hanafi *et al.*, "Dual-lockstep microblaze-based embedded system for error detection and recovery with reconfiguration technique," in *2015 Third World Conference on Complex Systems (WCCS)*, 2015, pp. 1–6.
- [33] A. B. De Oliveira *et al.*, "Lockstep Dual-Core ARM A9: Implementation and Resilience Analysis Under Heavy Ion-Induced Soft Errors," *IEEE Transactions on Nuclear Science*, vol. 65, no. 8, pp. 1783–1790, Aug. 2018. [Online]. Available: <https://ieeexplore.ieee.org/document/8401918/>
- [34] J. Yao *et al.*, "Dara: A low-cost reliable architecture based on unhardened devices and its case study of radiation stress test," *IEEE Transactions on Nuclear Science*, vol. 59, no. 6, pp. 2852–2858, 2012.
- [35] M. T. Sim *et al.*, "A dual lockstep processor system-on-a-chip for fast error recovery in safety-critical applications," in *IEEE IECON*, 2020, pp. 2231–2238.
- [36] J. Li *et al.*, "Duckcore: A fault-tolerant processor core architecture based on the risc-v isa," *Electronics*, vol. 11, no. 1, 2022.
- [37] L. Blasi *et al.*, "A RISC-V fault-tolerant microcontroller core architecture based on a hardware thread full/partial protection and a thread-controlled watch-dog timer," in *APPLEPIES*, 2019, pp. 505–511.
- [38] A. E. Wilson *et al.*, "Neutron radiation testing of fault tolerant risc-v soft processor on xilinx sram-based fpgas," in *IEEE SCC*, 2019, pp. 25–32.
- [39] D. A. Santos *et al.*, "A low-cost fault-tolerant risc-v processor for space systems," in *DTIS*, 2020, pp. 1–5.
- [40] P. R. Nikiema *et al.*, "Design with low complexity fine-grained dual core lock-step (dcls) risc-v processors," in *2023 53rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks - Supplemental Volume (DSN-S)*, July 2023.
- [41] —, "Impact of transient faults on timing behavior and mitigation with near-zero wct overhead," in *ECRTS 2023 - 35th Euromicro Conference on Real-Time Systems*, Vienna, Austria, July 2023.
- [42] N. I. Deligiannis *et al.*, "Towards the integration of reliability and security mechanisms to enhance the fault resilience of neural networks," *IEEE Access*, vol. 9, pp. 155 998–156 012, 2021.